DJFS : Directory-Granularity Filesystem Journaling for CMM-H SSDs

Seung Won Yoo, *Korea Advanced Institute of Science and Technology* (*KAIST*)*;* Joontaek Oh, *University of Wisconsin–Madison;* Myeongin Cheon and Bonmoo Koo, *Korea Advanced Institute of Science and Technology* (*KAIST*)*;* Wonseb Jeong, Hyunsub Song, Hyeonho Song, and Donghun Lee, *Samsung Electronics;* Youjip Won, *Korea Advanced Institute of Science and Technology* (*KAIST*)

https://www.usenix.org/conference/fast25/presentation/yoo

This paper is included in the Proceedings of the 23rd USENIX Conference on File and Storage Technologies.

February 25–27, 2025 • Santa Clara, CA, USA

ISBN 978-1-939133-45-8

Open access to the Proceedings of the 23rd USENIX Conference on File and Storage Technologies is sponsored by

DJFS : Directory-Granularity Filesystem Journaling for CMM-H SSDs

Seung Won Yoo* Joontaek Oh†* Myeongin Cheon* Bonmoo Koo*
Wonseb Jeong‡ Hyunsub Song‡ Hyeonho Song‡ Donghun Lee‡ Youjip Won*
*KAIST †University of Wisconsin-Madison ‡Samsung Electronics

Abstract

In this paper, we propose DJFS, *Journaling Filesystem with per-Directory Transaction*. By analyzing the file access patterns in eight popular applications, we find that most file update operations are centered around the associated directory. Based upon this observation, we propose that the journaling filesystem defines the transaction in per-directory basis. DJFS consists of three key ingredients: path-based transaction selection, transaction coalescing and transaction conflict resolution. Per-directory journal transaction successfully addresses the fundamental issues associated with improving the performance of the journaling filesystem: reduce the lock contention, reduce the transaction conflict, reduce the transaction lock-up, and parallelize the journal commit. DJFS improves the throughput by 4.5× in Varmail, 2.5× in MDTest, and 3.7× in Exim, compared to the state-of-the-art journaling filesystem, FastCommit.

1 Introduction

Protecting the filesystem under unexpected system failures has been of paramount interest to server vendors, storage vendors as well as application developers. Filesystem Journaling, a form of Write-Ahead-Logging [41], has been used as one of the most popular ways to protect the filesystem against the unexpected system crashes. There are two notable trends in modern computer system design. First, the number of cores increases rapidly. The modern server is equipped with as many as hundreds of cores [18,20] and the server is loaded with hundreds of VM images that run concurrently [52, 64]. Second, the storage device is getting quicker and faster. The newly introduced IO interface standards CXL (Compute Express Link) allows the CPU to access the storage device with block interface (.io protocol) as well as with cache-line granularity memory instruction (.mem protocol) [50]. The CXL-compliant CPUs [17, 43, 44] and devices are commercially available. They include not only CXL-based SSDs, but CXL memory modules [6, 11, 12], accelerators [3], and switches [14].

A large number of applications store and update the data on storage devices. They include key-value stores [9], relational databases [7], mail servers [27], and AI applications at edge devices [16]. As a number of these applications run on a single server that shares the same filesystem partition, the journaling module often becomes a performance bottleneck. This bottleneck becomes more serious as storage gets faster and as the number of applications exploiting the manycore increases.

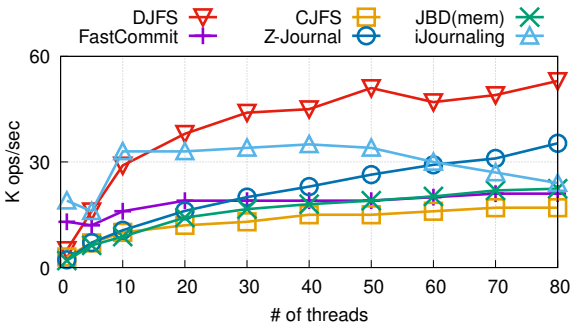


Figure 1: MDTest, Metadata Update Throughput (CMM-H Prototype [50], JBD(mem) is cache-line granularity JBD)

Several works have been proposed to address the performance issue of filesystem journaling [18,21,22,30–32,37–39, 47,48,51,54,56,62]. These works primarily focus on addressing (i) lock contention on the global journal structure [30,31], (ii) transaction conflicts among multiple concurrent transactions [32,38,47] and (iii) serial journal commits [22,37,48]. One important issue that has been overlooked in the earlier works is the overhead of transaction lock-up. In earlier journaling filesystem designs, transaction lock-up has not been a significant issue. The transaction lock-up is almost hidden under the long write latency of the storage device. As the storage gets quicker, the transaction lock-up begins to account for a larger fraction of the entire transaction latency. We carefully argue that the transaction lock-up should be handled with more care in designing the journaling filesystem.

In this work, we propose yet another journaling filesystem for the newly introduced CXL-based SSD, CXL Memory Module-Hybrid (CMM-H). CMM-H SSD distinguishes itself from the other block storage in three aspects. First, CMM-H SSD can be accessed in cache-line granularity with CXL.mem protocol as well as in block granularity with CXL.io protocol. Second, unlike the NVMe with Persistent Memory Region [45][1], CMM-H SSD provides cache-line granularity access to the entire storage area. Third, CMM-H SSD provides not only the low-latency but also high bandwidth through the block interface (CXL.io) [50]. Several filesystems have been proposed for byte or cache-line granularity-accessible storage device [24,25,65] or heterogeneous storage system with NVM [15,16,36,53,63]. Few of these works are suitable

*This work was done while the author was a graduate student at KAIST.

[1]Only small designated region, e.g. 4 MByte, of an SSD can be accessed with cache-line granularity

for the CMM-H SSD since they are not designed to exploit the hierarchical architecture of CMM-H SSD. The throughput of the CMM-H SSD highly depends on the built-in DRAM cache hit rate [66]. Our evaluation (section §2.3) shows that the random write throughput at cache-line granularity can vary by up to 74 times depending on the working set size. The existing works for NVM involve the random access on at least tens of GB of storage partition. The internal DRAM cache size of the CMM-H SSD is limited to a few GB at most.

We design the filesystem based on our analysis along the two axis; file update characteristics and cache-line granularity journaling. We examine the file update characteristics of eight widely used applications. We find the three key properties that are somewhat well-known and unsurprising; (i) Each application calls the file operations on its own dedicated directory, (ii) The application updates a series of files consecutively within the dedicated directory and (iii) Updating the contents of a file involves the creation and deletion of files. Despite these commonly observed and widely known characteristics of the file update pattern, we find that modern journaling filesystem design leaves substantial room for improvement in properly exploiting these characteristics. Second is the analysis of the performance benefit of using cache-line granularity journaling. For this analysis, we implement the cache-line granularity (64Bytes) journaling in JBD [60]. We examine the performance under four benchmarks, `varmail`, `MDTest`, `exim`, and `fillsync` in `RocksDB`. While the journal commit latency decreases by 50%, we find that the throughput gain achieved by using cache-line granularity journaling instead of block granularity is marginal. The benchmark throughput increases by less than 20%. We find that the transaction lock-up time becomes 70% longer as we use cache-line granularity journaling.

Based on our observations, we propose to define a journal transactions on a per-directory basis. We identify three key challenges in realizing per-directory transactions; (i) a file with multiple links, (ii) multi-directory transactions, and (iii) an order-preserving journal commit. DJFS consists of three technical ingredients each of which is designed to address the above-mentioned technical challenges; *path-based transaction selection*, *transaction coalescing*, and *transaction conflict resolution*. Our key contributions are as follows.

- We discover that the performance gain of using finer-grained journaling against block granularity is marginal. The root cause of this performance inefficiency is the increased transaction lock-up overhead. We find that a reduction in the commit latency results in the increase in the transaction lock-up time.
- By analyzing the eight widely used applications, we identify three common file update patterns. Based on this analysis, We carefully argue that a *directory* is a good unit for filesystem journaling. We develop a journaling filesystem that uses a directory as a unit of journal transaction.

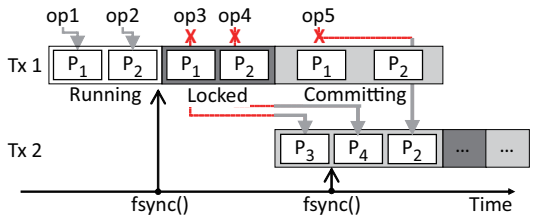


Figure 2: Concurrency Control in JBD

- We identify three key technical challenges in using the directory as the filesystem journaling granularity and develop the techniques to address three challenges.

We implement DJFS on EXT4 with Linux Kernel 5.18. Compared to the state-of-the-art journaling technique, Fast-Commit [51], DJFS renders 4.5× performance in `Varmail`, 2.5× performance in `MDTest`, and 3.7× performance in `Exim`, while achieving similar throughput in `RocksDB-fillsync`.

2 Background

2.1 Journaling in EXT4

EXT4 filesystem employs a journaling mechanism [60] with transactions in three states: running, committing, and checkpointing. JBD maintains one running transaction, one committing transaction, and multiple checkpointing transactions. A transaction begins in the running state. It contains a set of page cache entries that contains the updated metadata. When the JBD thread starts committing, the transaction status changes to committing. This occurs either when the periodic commit timer expires or `fsync()` is called. The journaling thread logs the transaction header and page cache entries to the journal area, ensuring their durability before writing a commit block. Once committed, the transaction's status changes to checkpointing, during which page cache entries are reflected in their original location in the background.

Four main reasons hinder journaling throughput: (i) lock contention on the transaction [30,31,54], (ii) transaction lock-up [47], (iii) transaction conflict [32,47], and (iv) serial journal commit.

(i) Lock Contention. A running transaction can carry the results of multiple file operations [26,60]. This is called a compound transaction. The compound transaction is inevitably subject to lock contention [30,54]. In Figure 2, when the file operations `op1` and `op2` (e.g.,`creat()`) are called, the insertions of updated page cache entries (`p1` and `p2`) are serialized.

(ii) Transaction lock-up. When the application calls `fsync()`, the JBD thread begins the transaction commit only after all ongoing file operations in the running transaction return. During this period, the filesystem stops issuing the journal handle, blocking the new file operations that update the metadata. This interval is known as the transaction lock-

up period. In Figure 2, file operations `op3` and `op4` are blocked due to the transaction lock-up. Once the transaction lock-up period is over, `op3` and `op4` can insert the page cache entries into the newly created running transaction, `Tx 2`.

(iii) DMA Transfer and Transaction Conflict. After the transaction lock-up period is over, the JBD thread changes the transaction state from running to committing and initiates a DMA transfer of the transaction's page cache entries. These entries cannot be modified during the transfer. File operation threads attempting to update these entries are blocked due to transaction conflicts [62], until the transaction becomes durable. Once the transactions become durable at the journal region, the JBD thread wakes the blocked threads. In Figure 2, file operation `op5` tries to update the page cache entry `p2`, but since it is part of the committing transaction `Tx 1`, it remains blocked until `Tx 1` becomes durable.

(iv) Serial Journal Commit. JBD commits the transactions serially. JBD thread commits a running transaction when there is no ongoing commit. If there is a committing transaction, JBD thread postpones the commit of the running transaction until the committing transaction becomes durable.

2.2 Scalability in Filesystem Journaling

Several approaches have been proposed to mitigate the scalability bottleneck in journaling [18, 21, 22, 30–32, 37–39, 47, 48, 51, 54, 56, 62]. These approaches focus on addressing lock contention [30, 31], transaction conflicts [32, 38, 47], and serial journal commits [22, 37, 48]. As we will demonstrate in §3.2, the transaction lock-up is negligible with block devices but becomes non-negligible with CMM-H SSDs. However, among the proposed solutions, only CJFS [47] specifically addresses this issue. Many of these methods allocate multiple running transactions [18, 30, 32, 38, 39, 48] or focus on committing transactions [47, 62]. These strategies can be further categorized by how transactions are allocated, including per-core, per-partition, per-file, or through concurrent commit.

Per Core Transaction. ScaleFS [18] and Z-journal [32] allocate the running transaction in per-core basis. However, in the per-core basis approach, the transaction conflict occur frequently, and it introduces version control overhead.

Per Partition Transaction. IceFS [39] and SpanFS [30] divide the filesystem into multiple physical partitions and allocate the running transaction to each of them. A partition's lock-up does not block the other partition's file operation. However, in the per-partition basis approach, the on-disk layout should be changed.

Per File Transaction. iJournaling [48] allocates the transaction in per-file basis. However, the unit of a transaction is too small to benefit from the group commit [26, 47, 60].

Concurrent Commit. CJFS [47] is the only work that addresses the extended lock-up intervals caused by transaction conflicts. CJFS mitigates this by adopting the trylock mechanism; it stops the lock-up when a conflict is detected. However, it results in increased commit latency.

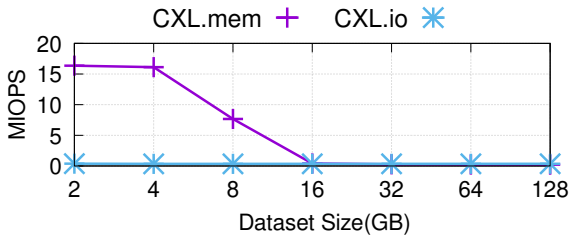


Figure 3: Random write Throughput according to the dataset size (88Core, CMM-H Prototype, IO size: 64Bytes for CXL.mem and 4 KB for CXL.io)

2.3 CMM-H: The Memory Semantic SSD

The CMM-H (CXL Memory Module Hybrid) [50] is an SSD that supports I/O based on the CXL.mem protocol [33, 50, 66]. CMM-H supports both block-level I/O and cache-line-level I/O. Block-level I/O (CXL.io) provides high bandwidth, while cache-line-level I/O (CXL.mem) offers low latency. According to our measurements, the bandwidth of CMM-H is 4 GB/s and 2 GB/s for block IO and cache-line level IO, respectively. The latency of CMM-H is 10 us and 0.6 us for block IO and cache-line level IO, respectively. CMM-H exposes NVMe storage to the host. Additionally, CMM-H exposes HDM (Host-managed Device Memory) to the host in the same size as NVMe storage. The HDM exposed by CMM-H is mapped 1:1 with the exposed NVMe storage. The host performs block-level I/O using NVMe commands. The host can perform cache-line level I/O by registering the HDM to the V2P mapping and issuing `load/store` instructions.

CMM-H provides cache-line granularity access to the entire storage space. This homogeneous storage space of CMM-H is implemented with the hierarchical structure of CMM-H where the upper layer (DRAM) caches the contents of the lower layer (NAND). Thus, the access latency to an LBA can vary widely depending upon whether a given LBA is cached in the DRAM or not internally. In other words, the throughput is sensitive to the built-in dram cache hit rate [66]. Figure 3 illustrates the results. The write throughput of CXL.mem is 16 MIOPS with the 2 GB dataset size. It decreases by 50% when the working set size increases to 8 GB. When accessing the CMM-H via the CXL.mem protocol, it is essential to either reduce the dataset size or enhance locality to increase the built-in DRAM cache hit. Existing works on persistent memory filesystems have typically overlooked access locality [24, 25, 36, 55, 63, 65, 67]. Specifically, prior PM filesystems exhibit two properties: they perform random writes across the entire PM space [24, 25, 65], and they typically operate with dataset sizes in the tens of GB [36, 63, 67]. These characteristics make it challenging for existing works to achieve optimal performance on CMM-H.

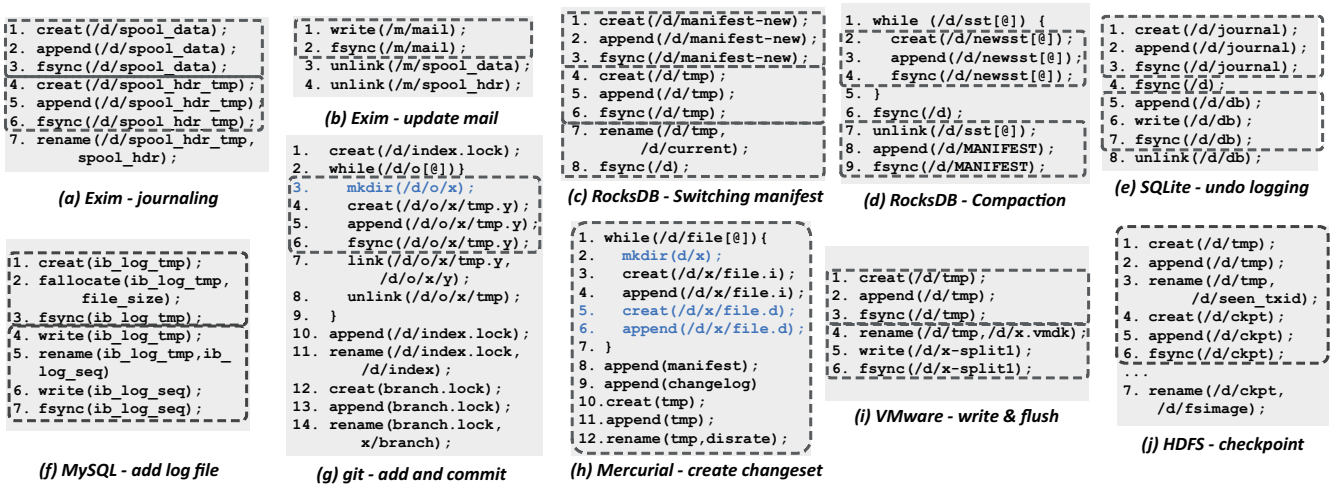


Figure 4: Application File Operation Pattern from [1,8,10,28,46,49,57] : a set of file operation commit with `fsync()` is bordered, blue is marked for conditionally called file operation.

3 Characterization Study

3.1 Application's File Update

To ensure the crash consistency of the user data, the applications adopt atomic rename [28, 49, 57], multi-file transaction [46, 49], etc. Atomic rename, multi-file transaction and etc. consist of the multiple file updates and a series of `fsync()` calls. Atomic rename and multi-file logging put non-negligible pressure on the modern journaling filesystem. We analyze eight widely used applications and examine the common patterns in the file update behaviors of these applications. We find three essential ingredients in file update behaviors.

- **Dedicated Directory (Property D)** An application works on its own dedicated directory. If an application consists of multiple threads, each thread operates in a separate directory.
- **Directory Update (Property U)** Updating the contents of a file involves the creation and deletion of files. Updating the file contents also requires updating the parent directory.
- **Shared Directory (Property S)** The application updates the multiple files that belong to the same directory.

We carefully argue that despite these widely observed common characteristics, the existing journaling filesystems fail to exploit these common characteristics in designing the journaling mechanism and leave substantial room for improvement. The characteristics of the file updates in each application can be summarized as follows:

Storing email in Exim Exim [27] is a widely used email client on Unix systems. Exim has its own dedicated directory (Property D). Figure 4(a) and Figure 4(b) show Exim's file operation patterns. When receiving mail, Exim updates both the header and data files (Property S), and these files are newly created (Property U).

Compaction in RocksDB RocksDB [9] works in its own dedicated directory (Property D). Figure 4(c) illustrates RocksDB's log file replacement, and Figure 4(d) depicts the file operation pattern during merge sort. The results of the merge sort are stored in new files (Property U). If the merge sort result is large, it is split across multiple files (Property S). When replacing log files, RocksDB creates new log files (Property U). The newly created file names are stored in other files using an atomic rename operation (Property S).

Transaction in SQLite SQLite [13] operates within its own dedicated directory (Property D). Figure 4(e) illustrates the file operation pattern of SQLite's undo logging [28, 46, 49]. SQLite updates the journal file and the database file in order (Property S). The journal file is created before the database file is updated. The journal file is removed after the database file is updated (Property U).

Create Log File in MySQL MySQL [7] is a server-oriented RDBMS that performs operations in its own directory (Property D). Figure 4(f) shows the log file creation. When creating log files, MySQL uses atomic rename (Property U).

Source Commit in Git Git [2] operates within its own directory in the repository (Property D).Figure 4(g) shows the source commit operation in Git. When changes occur in the repository, Git creates a blob object (Property U). Since a blob object is created for each file change, multiple objects are often generated during a source commit (Property S).

Create Changeset in Mercurial Mercurial [5], a version control system, operates within its own directory in the repository (Property D). In Figure 4(h), Mercurial generates the log files for each file (Properties U and S). Since `fsync()` is not called, updated metadata is asynchronously reflected on

Kops/s	Varmail	MDTest	Exim	RocksDB
JBD(blk)	38.0	18.5	4.6	30.9
JBD(mem)	42.5	22.3	6.1	34.5

Table 1: Throughput at 80 threads, blk: Block granularity journaling, mem: cacheline granularity journaling, 80 threads

us	Varmail	MDTest	Exim	RocksDB
JBD(blk)	131	273	292	65
JBD(mem)	62	140	288	2

Table 2: Disk write time during journal commit, 80 threads

disks.

Updating VM Disk Image in VM Hypervisor Figure 4(i) shows VMware's operation for updating a virtual disk file. If the virtual disk file is small, VMware updates the file using atomic rename (Property U). If the virtual disk file is split into multiple parts, the descriptor file is updated (Property S).

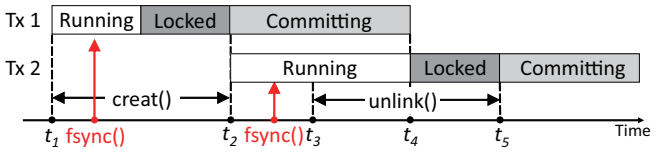
Journal Checkpoint in HDFS HDFS [19] performs the file and database operations in its own directory (Property D). Figure 4.(j) shows the periodic journal checkpoint in HDFS. During checkpointing, HDFS updates the last transaction ID, checksum, and the filesystem image using atomic rename (Properties S and U).

3.2 Effect of Finer Granularity Journaling

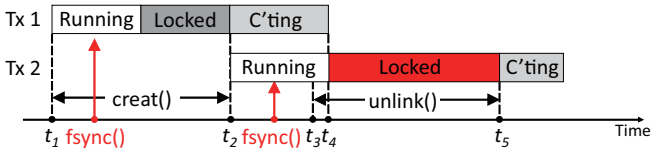
We examine the performance effect of naively adopting finer granularity journaling. We modify the JBD to maintain the log record in cache-line granularity (64Byte). The cache-line granularity journaling is implemented on the CMM-H prototype. Cache-line granularity JBD writes the transaction to the disk by `store` instruction (`.mem` interface). Transaction structure of the cache-line granularity JBD and DJFS are the same. This will be detailed in section Figure 5.1. We use four benchmarks; `varmail` [40], `MDTest` [4], `Exim` [27], and `fillsync` at `RocksDB` [9]). The server is equipped with 88 Core (2×44) Intel Sapphire Rapids and a 2 TB CMM-H prototype.

The performance improvement of using finer journaling granularity from 4 KByte to 64 bytes is not as substantial as we expected. The performance improvement is merely 20%. The results are in Table 1. The performances of cache-line granularity JBD correspond to 1.1× for Varmail, 1.2× for MDTest, 1.3× for Exim, and 1.1× for RocksDB, respectively, compared to those of block granularity JBD.

We find that the finer granularity journaling (with a high-performance storage interface) is subject to more significant transaction lock-up overhead. In block granularity journaling, the transaction lock-up interval accounts for 6% of a journal commit latency. With cache-line granularity journaling, the transaction commit latency decreases by 35%. However, the



(a) Coarse Grained Commit with Block Device



(b) Fine Grained Commit with CMM-H SSD

Figure 5: Comparison of Lock-Up Period Transaction commit is triggered by the `fsync()` marked by red arrow.

transaction lock-up accounts for a more significant fraction of the transaction commit, rising to 18%. Due to the serialized journal commits in JBD, a shorter journal commit latency results in a longer lock-up time. JBD locks up the next running transaction only after the current committing transaction has persisted. As a result, a decrease in journal commit latency causes the lock-up to start earlier. However, the latency of in-memory file operations remains constant, regardless of journal commit latency. Consequently, the lock-up end time stays unchanged, leading to an overall increase in transaction lock-up time. During the lock-up period, all metadata updates in the system are blocked because JBD maintains a single running transaction, coalescing all updated metadata into this transaction. As a result, an increase in lock-up time directly reduces system throughput.

4 Per Directory Journal Transaction

4.1 A Concept

```
1. creat(/d/newt);
2. append(/d/newt);
3. fsync(/d/newt);
4. fsync(/d); Skipped!
5. unlink(/d/oldt);
6. creat(/d/MANI-NEW);
7. append(/d/MANI-NEW);
8. fsync(/d/MANI-NEW);
```

Tx 1

Tx 2

Figure 6: Transaction granularity in DJFS, a fraction of file operations of RocksDB

In this work, we propose yet another journaling filesystem with better filesystem journaling scalability. We establish three objectives in designing the journaling module. First, we define the multiple running transactions so that they can be

updated independently and can be committed concurrently. Second, we minimize the conflict among the running transactions to prohibit them from being serialized unnecessarily. Third, we minimize the transaction lock-up overhead associated with committing a running transaction to prohibit the filesystem operations from being blocked for significant period of time.

Achieving these design objectives boils down to defining multiple running transactions in the system that are as small as possible where each of them maintains an exclusive set of updated metadata.

A fair amount of works have been proposed to improve the performance of the filesystem journaling. A few works carry the results of multiple file operations in a single journal transaction, e.g. running transaction in EXT4 [60], committed item list in XFS [56], or per-partition transaction [30, 39]. These works improve the journaling throughput through group commit. However, they suffer from significant transaction lock-up overhead since a large number of filesystem operations can share a single transaction. Another body of works define the transaction in smaller units, e.g. in a per-file basis [48], on a per-operation basis [18, 34], or in a per-commit basis [47, 62]. These works successfully reduce the transaction lock-up overhead in committing a single transaction. Often, the reduction in the compound degree in the journal transaction offsets the benefit of reducing the transaction lock-up overhead. A few works statically allocate multiple transactions, e.g. per-core basis [18, 32], per-partition basis [30, 39]. These works do not take into account the file update characteristics in creating the journal transaction. As a result, the journal transactions are subject to conflicts frequently, i.e. the transactions may attempt to update the same block. Transaction conflict is an important cause of scalability failure in journaling [32, 47, 62].

In this work, we propose to define a journal transaction in *directory* granularity. There are two key benefits. First, we can minimize the transaction conflict and transaction lock-up when we define the journal transaction in per-directory basis. Based upon Property D (dedicated directory), each thread works on its directory exclusive from the rest. When a transaction is locked up for journal commit, the other threads are not affected and can continue their file operations. Second, using per-directory transactions, we can carry multiple file operations that are related to each other though the transaction granularity becomes finer. Most file update operations update not only the set of files but also the associated parent directory (Property U and S). By defining a transaction in per-directory basis, a running transaction can contain a proper set of file operations that are related to each other.

In DJFS, the updated metadata is inserted into the running transaction of the parent directory inode of the last component of the path string. Once the directory inode is found, its running transaction can be obtained through the running transaction pointer of the directory inode. If a file operation only updates an inode, e.g. `chmod(/d1/f1,777)`, the updated

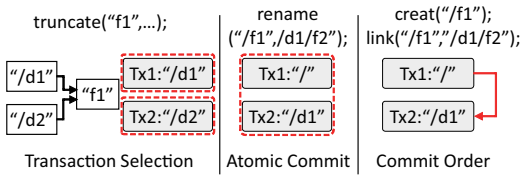


Figure 7: Three technical challenges

inode is inserted into the running transaction of directory `d1`. If a file operation updates an inode and its index block, e.g. `truncate(/d1/f1)` updated inode and the index block are inserted into the running transaction of directory `d1`. There are two file operations that require elaboration; `mkdir(path)` and `link(oldpath, newpath)`. For `mkdir(path)`, we inserts the newly created directory inode to its parent directory's running transaction. In the case of `link(oldpath,newpath)`, the updated directory is the directory that owns the `newpath` directory entry. Thus, even though the inode pointed by the `oldpath` and `newpath` is the same, the updated metadata is inserted into the parent directory of `newpath`.

4.2 Challenges

We identify three technical challenges in realizing a per-directory transactions: (i) updating a file with multiple links, (ii) updating the multiple directories in a transactional fashion, and (iii) resolving the transaction conflict.

Updating a File with Multiple Links. An inode can be referred to by more than one directory entry as shown in the left side of Figure 7. In a per-directory transaction, there can be multiple running transactions whose directory entries refer to the same inode. The file with the multiple hard links can be updated. The journaling module needs a mechanism for how to handle the updates to the file with multiple links.

Transactional Update on Multiple Directories. A single file operation can update more than one directory as shown in the middle side of Figure 7. For example, `rename(old,new)` may update as many as four directories [58]. Many filesystems [30, 32, 35, 56, 60] support atomicity in `rename()` with few exceptions [18, 23]. When we define the transaction in per-directory basis, a single file operation may create multiple transactions, each of which is committed independently. In a per-directory transaction, we need a mechanism to handle multiple directory updates in a transactional manner.

Resolving the Transaction Conflict. A file operation may update the metadata object that already belongs to another transaction. The transactions under the conflicts need to be properly synchronized to ensure the crash consistency of the filesystem. The right side of Figure 7 illustrates this scenario. When a thread calls `link(/f1,/d1/f2)`, the updated metadata should be inserted into the running transaction of `d1`. However, the inode pointed by `f1` is already part of the root directory's

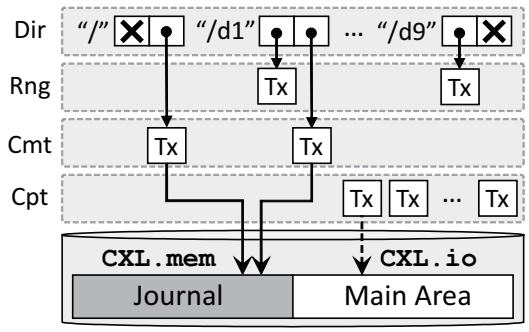


Figure 8: An overview of DJFS (Dir : directory, Rng : running, Cmt : committing, Cpt : checkpointing)

transaction due to the previous `creat(f1)`. In this case, the transaction for `d1` should either be merged with the transaction for the root directory or committed after the root directory's transaction.

5 DJFS

5.1 Design Overview

In this work, we propose a journaling filesystem that is based per-directory granularity journaling. DJFS allocates the transaction in per-directory basis. Each directory can have up to one running transaction and one committing transaction. A directory inode has two transaction pointers, one for running transaction and the other for committing transaction.

DJFS journals only file metadata, e.g. inode, file map, and directory entry [48, 51, 69]. The filesystem metadata is reconstructed during the crash recovery. There are two major benefits. First, we can reduce the frequency of transaction conflicts and increase the concurrency degree of journal commit. Second, we can reduce the size of a transaction and journal commit latency.

Metadata changes for directories and their associated files are recorded in the log records. A log record represents the updated contents in a 64-byte unit. When an application performs file operation, it identifies the directory and inserts the log records into the directory's running transaction. If the directory does not have running transaction, DJFS allocates a running transaction for the directory. The running transaction is added to the running transaction list at the filesystem. The running transaction list is used for periodic journal commit.

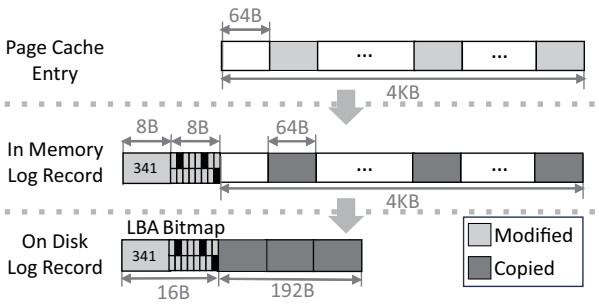


Figure 9: Bitmap-based log representation, B: Bytes

The journal transaction can be committed either periodically or by `fsync()`. Before a transaction commit begins, the transaction is removed from the running transaction list. Multiple transactions may be committed simultaneously if several `fsync()`'s are called within a short time interval. Committed transactions are registered at the checkpoint transaction list and the *kworker* thread performs checkpoints either periodically or when there is no free space in the journal region.

5.2 Data Structures

Log Record. We design the logging scheme to fully exploit the cache-line access granularity of CMM-H. DJFS journals three types of metadata; inode, index block, and directory block. For inode, DJFS uses inode granularity (256 bytes) physical logging. When an inode is updated, the filesystem creates a copy of the updated inode and inserts it into the running transaction. For index block and directory block, it uses a cache-line granularity (64 bytes) differential logging. DJFS partitions the index block and directory block with cache-line granularity (64 bytes) and uses a bitmap to represent the updated location within a block. When an index block or a directory block is updated, the filesystem allocates a shadow page, sets the bitmap entries for the updated region(s), and records the updated contents on the shadow page. A log record (both in-memory and on-disk) consists of a log descriptor and an update. Log descriptor contains the type of log; inode, index block, or directory block, and the disk location of the associated log. For the inode log, the disk location field contains the inode number. For index block or directory block, the disk location contains the LBA of the updated disk block and the bitmap. When committing a journal transaction, only the updated regions in the log record are written to the journal region. Figure 9 illustrates the in-memory representation and on-disk representation of the log record for an index block. With differential logging, we can save time for updating the in-memory log record and save disk space in the journal area.

In-memory Transaction. The in-memory transaction maintains a set of log records, a set of associated page cache entries, inode number and the number of outstanding file operations in the running transaction (operation count). Using the page cache entries, the checkpoint module can avoid read-modify-write in checkpointing the log records and can simply perform blind write. Figure 10 illustrates the in-memory layout of directory transactions.

On-disk Transaction. The on-disk transaction structure is devised for cache-line granularity logging. An on-disk trans-

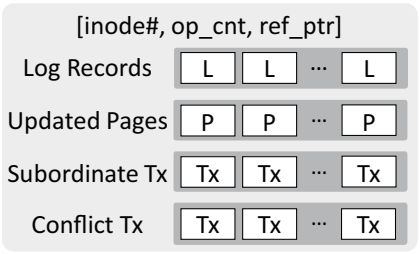


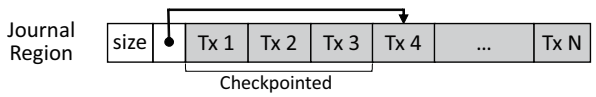
Figure 10: Structure of in-memory transaction in DJFS

action consists of a transaction header, a set of log records, and a commit record. The header contains the transaction ID, the number of log records, and the transaction size. The commit record is the 8-byte magic number. Figure 11b illustrates the on-disk transaction layout.

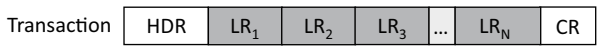
Journal Region. The journal region is a single circular array of transactions. The journal region of DJFS should be accessible at the speed of CXL-attached memory. We make the size of the journal region relatively small so that it can be cached at the internal DRAM of a CMM-H SSD. Figure 11a illustrates the journal region in DJFS.

5.3 Path-based Transaction Selection

A file can have more than one link (hard link) and subsequently can have more than one parent directory. If a file with more than one parent directory is updated, the filesystem needs to select either of the two parent directories at whose transaction it inserts the updates. In DJFS, a file operation selects the running transaction based on the path supplied as an argument. We call it a *Path-based Transaction Selection*. A POSIX file operation can take either the path string or the file descriptor as its argument. When the file operation receives a file descriptor as the argument, it can identify the path of the file descriptor by traversing the directory entry from the associated `struct file` object. If the file operation argument is a relative path, its parent directory inode can be obtained through the current working directory. Figure 12 shows an example of path-based transaction selection. The file `f1` is associated with two directories, directory `d1` and directory `d2`.



(a) On Disk Journal Layout



(b) On Disk Transaction Layout

Figure 11: On-Disk layout of DJFS, HDR: transaction header, Tx_i: transaction i, LR_i: log record i, CR: Commit Record

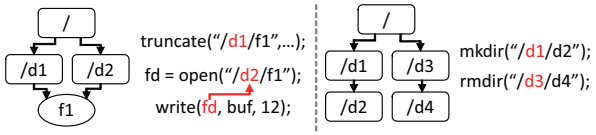


Figure 12: Path based Transaction Selection; selected directories are marked as red.

For `truncate("/d1/f1",...)`, the updated metadata for `f1` is inserted into the running transaction of directory `/d1`. For `mkdir("/d1/d2")` and `rmdir("/d3/d4")`, the updated metadata are inserted into the running transactions of `d1` and `d3`, respectively.

5.4 Transaction Coalescing

A single file operation may update two files that belong to two different directories. If the updates are stored at separate transaction, the failure-atomicity of a file operation may be compromised. To address this issue, we developed a technique called *Transaction Coalescing*.

Transaction coalescing merges the two transactions into one. In transaction coalescing, one of the two transactions is designated as a master transaction and the master transaction maintains the pointers to the subordinate transactions. A master transaction can have one or more subordiate transactions. The filesystem chooses the transaction with the lower inode number of the two transactions as the master transaction. All transactions in a coalesced transaction should be committed atomically. When a subordinate transaction is called with `fsync()`, the commit request is redirected to the master of the subordinate transaction. In committing a coalesced transaction, journaling module logs all log records at the master transaction but also at the subordinate transactions.

Transaction coalescing is done with the following steps. First, DJFS acquires the spinlocks for the inodes of both the master and subordinate transactions [2]. To avoid deadlock, DJFS acquires spinlocks in ascending order of inode numbers. Second, DJFS updates the operation count of the master transaction by adding the operation count of the subordinate transaction. This prevents the master transaction from being committed before file operations associated with the coalesced transaction return. Third, DJFS redirects new file operations from the subordinate transaction to the master. To implement this, a reference pointer is added to the transaction structure. When transactions are coalesced, DJFS updates the reference pointer of the subordinate transaction to reference the master transaction, ensuring that all subsequent file operations are redirected to the master transaction. Fourth, the subordinate transaction is inserted into the master transaction's subordinate list. Then, DJFS releases all the inode spinlocks acquired.

[2] DJFS introduces a new spinlock in the in-memory inode structure.

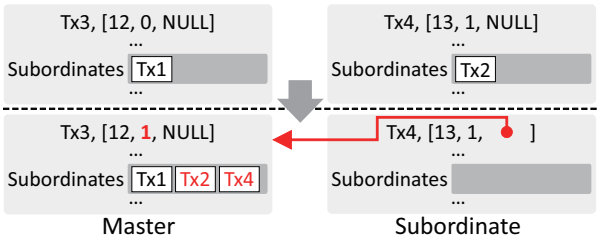


Figure 13: Transaction coalescing in `rename()`

If one or both of the transactions being coalesced are already subordinate transactions of the existing coalesced transactions, DJFS coalesces the master transactions of them. If an existing master transaction becomes a subordinate transaction as a result of coalescing, all existing subordinates are moved to the new master's subordinate list.

Figure 13 illustrates an example of transaction coalescing. The `rename("/d1/f1", "/d2/f2")` operation updates the inodes `d1`, `d2`, and `f1`, as well as the directory entries of `d1` and `d2`. DJFS coalesces the running transactions of `d1` and `d2` before updating the metadata. In this example, let us assume that `d1` becomes the master and `d2` becomes a subordinate. DJFS carries out the coalescing in three steps. First, it adds the operation count of `d2`'s running transaction to the master. Second, it updates the reference pointers of `d2`'s running transaction and any subordinate transactions to point to `d1`'s running transaction. Third, DJFS inserts `d2`'s running transaction and its subordinates into the subordinate list of `d1`.

5.5 Resolving the Transaction Conflict

In DJFS, there can be a number of running transactions and committing transactions in the system. A file operation can update a metadata whose shadow copy is already in another transaction. We call it a transaction conflict.

When a file operation updates the metadata, it examines if there is a transaction conflicts and resolves the conflict if there occurs one. DJFS detects the transaction conflict as follows. In DJFS, each inode has two *container transaction* pointers. The container transaction of an inode is a transaction where the shadow copy of its metadata belongs. We maintain two container transaction pointers for each inode: one for a running transaction and the other for a committing transaction. These pointers are set when the associated metadata is inserted at the running transaction or when the running transaction is committed. When the transaction is committed, the associated pointers becomes NULL. Before updating the metadata, a file operation examines the container pointers for conflicts. If the filesystem finds that any of the container pointers is not NULL, the filesystem detects the conflict.

When there is a transaction conflict, DJFS resolves the transaction conflict so that DJFS can guarantee the crash consistency. A transaction can conflict with more than one transaction. The filesystem runs a conflict resolution routine for each transaction which it conflicts with.

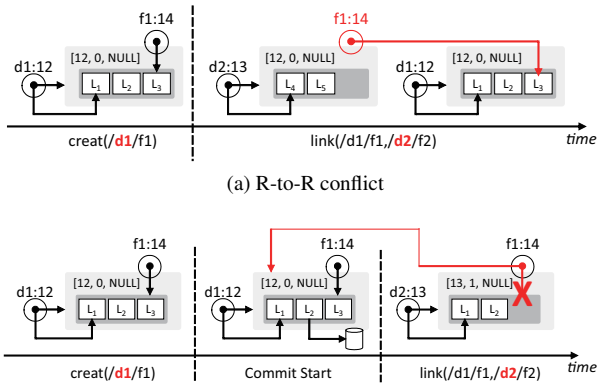


Figure 14: Transaction conflict of inode `f1`

There are two types of transaction conflict: running transaction to running transaction conflict denoted by R-to-R, and running transaction to committing transaction conflict denoted by R-to-Cmt. The former and the latter types of conflicts are resolved by a transaction coalescing and an ordered commit, respectively.

R-to-R conflict. R-to-R conflict occurs when one transaction tries to update the metadata that belongs to another running transaction. If the filesystem detects an R-to-R conflict, the filesystem coalesces the two transactions and updates the existing log record for the conflict operation. Due to transaction coalescing for R-to-R conflicts, at most one running transaction and one committing transaction can hold the shadow copy of the updated metadata. In Figure 14a, a single thread calls `creat(/d1/f1)` and `link(/d1/f1,/d2/f2)` in order. When the inode `f1` is created by `creat(/d1/f1)`, its running transaction container is set to the address of the running transaction of `d1`. When the inode `f1` is updated by `link(/d1/f1,/d2/f2)`, the running transaction container is already set but not the same as the address of the running transaction of `d2`. Thus, the two transactions are coalesced.

R-to-Cmt conflict. R-to-Cmt conflict occurs when the file operation updates the metadata that belongs to the committing transaction. If the filesystem detects an R-to-Cmt conflict, the filesystem delays committing the running transaction till the committing transaction under conflict is made durable at the storage. This is to ensure that the running transaction and committing transactions under R-to-Cmt conflict are committed in order. If the filesystem detects an R-to-Cmt conflict, the filesystem inserts the committing transaction to the conflict list of the running transaction. When the filesystem commits a transaction, it checks the conflict transaction list. The commit is delayed until the conflict list becomes empty. In Figure 14b, a thread calls `creat(/d1/f1)` and `link(/d1/f1,/d2/f2)` in order. Another

thread calls `fsync(/d1/f1)` after `creat(/d1/f1)` but before `link(/d1/f1,/d2/f2)`. When `creat(/d1/f1)` is called, the running transaction container of `f1` refers to the running transaction of `d1`. Later, when `fsync(/d1/f1)` is called, the committing transaction container of `f1` is set to its running transaction container. A `link(/d1/f1,/d2/f2)` updates the link count of inode `f1` and inserts its update to the running transaction of `d2`. Please note that the inode pointed by both `f1` and `f2` is same. However, the thread that calls `link(/d1/f1,/d2/f2)` finds that the committing transaction container is already set. The transaction pointed by the committing transaction container of `f1` is inserted into the conflict list of the running transaction of `d2`.

6 Implementation

6.1 Metadata Update

In DJFS, the metadata update consists of four phases: (i) acquire the lock with associated objects, (ii) reserve journal area, (iii) select a running transaction, and (iv) insert log records. An application thread first acquires the locks associated with objects (e.g., inode read/write semaphore), and it reserves the journal area not to abort the transaction during commit. After reserving the journal area, the application thread selects a running transaction. If the selected running transaction is locked, the application thread is blocked till the locked transaction becomes a committing transaction. Then, it creates and inserts a new running transaction to the running transaction list and attaches it to the directory inode. Subsequently, it checks whether the conflict occurs and handles it if needed. Then, it increments the outstanding operation counter, and it inserts the page cache entries and the log records into the running transaction. After insertions, the application thread decrements the operation counter and releases the lock associated with the object.

6.2 Transaction Commit

A transaction is committed with the following steps. First, the commit thread, i.e., `fsync()` thread or the kworker thread, checks if the associated directory has a committing transaction. If there exists an ongoing commit, it waits until the ongoing commit finishes. Second, when the ongoing journal commit finishes, the filesystem stops issuing the journal handle for the given transaction, and waits until all outstanding file operations return. It removes the transaction from the running transaction list. Then, the commit thread changes the state of the transaction from running to committing. After changing the state of the transaction, the transaction pointer of the directory inode is updated; The committing transaction pointer of the directory inode is set to the address of the currently committing transaction, Then, the container transaction pointers of the inodes belonging to the transaction are updated. Third, the commit thread allocates the journal area and a transaction ID for the journal transaction. In DJFS, while we allow multiple threads to commit the transactions in parallel, this procedure is serialized among the commit threads as in [39]. Fourth, the commit thread stores the log records at the designated location. When the commit thread ensures that the log records are made durable, it writes the commit record at the end of the transaction. Finally, the log records belonging to the transactions are then released. If the page cache entry does not belong to the other transaction, its write-back is reserved. Then, it changes the state of the transaction to the checkpointing.

6.3 Transaction Checkpoint

DJFS checkpoints the transaction either when the journal region lacks of free area or when the checkpoint thread wakes up periodically. In DJFS, a file operation can update the metadata that is in the page in the checkpointing transaction. In this case, the page contains the uncommitted update. Thus, when the filesystem blindly checkpoints the page cache entry, the results of a running transaction can be partially reflected on the disk, compromising the atomicity of a transaction. To prevent this situation, the checkpoint thread temporarily stops creating the running transactions and commits all running transactions when it starts the transaction checkpoint. Subsequently, the checkpoint thread dispatches dirty metadata pages and resumes the transaction creation. After the dirty metadata pages are persisted in place, the blocked file operation can continue.

6.4 Crash Recovery

Crash Recovery The recovery procedure works as follows. First, the recovery module scans the journal region and identifies the transactions that are committed but yet to be checkpointed. Second, it checkpoints the committed transactions. Third, it reconstructs the filesystem metadata at the time of filesystem crash [48, 51].

Crash Consistent Counter When the system crashes during the recovery, counter metadata reconstruction requires scanning the entire filesystem or whole block group. DJFS journals the counter during checkpoint to address this issue. During checkpoint, DJFS updates shadow counters instead of original counters, recording the last checkpointed transaction ID + 1. After updating the journal head, the original counter can be updated. Updating journal head indicates that the shadow counter is committed. If a crash happens during checkpoint, DJFS replaces the original counter with the shadow counters if the recorded transaction ID is the transaction ID pointed by the journal head.

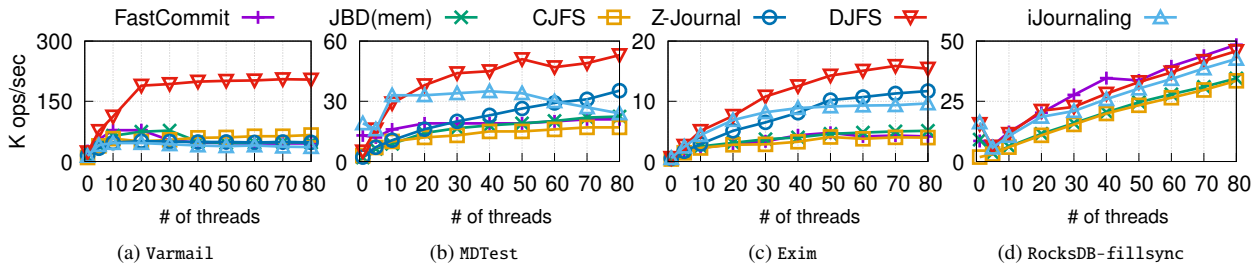


Figure 15: Throughput: JBD, CJFS, Z-Journal, FastCommit, iJournaling and DJFS

7 Evaluation

7.1 Experiment Setup

We implement DJFS over EXT4 on Linux 5.18. We compare DJFS against five state of the art journaling filesystems; cache-line granularity JBD (denoted as JBD(mem)), FastCommit [51], Z-Journal [32], CJFS [47], and iJournaling [48]. In the experiment, we use a 2 TB CMM-H prototype. The server has 88 cores (two Sapphire Rapids CPUs, each with 44 cores, 128 GB Memory). We use four popular macro benchmarks for the study: `Varmail` [40] `MDTest` [4], `Exim` [27], and `RocksDB-fillsync` [9]. We were not able to evaluate Z-Journal on RocksDB due to deadlock.

For fair comparison, we port EXT4, iJournaling, and FastCommit over CXL.mem interface to commit the journal transaction in cacheline granularity (64Byte). We do not port Z-journal and CJFS over CXL.mem interface. Z-Journal requires a prohibitively large journal area (172 GByte in total, 1 GByte per logical core), that cannot fit on the internal DRAM of CMM-H device. As observed in Figure 3, when the working set size exceeds the DRAM cache size of CMM-H, the benefit of using cache-line granularity access does not exist. CJFS is built on the order-preserving I/O stack [62] which is designed for the block interface. The sizes of the journal regions are set to their default values as follows; JBD(mem) at 1 GB, CJFS at 1 GB, Z-Journal at 172 GB, FastCommit at 16 MB, iJournaling at 1.1 GB (1 GB for JBD and 100 MB for per-file transaction), and DJFS at 100 MB.

7.2 Manycore Scalability

We evaluate the throughput of the `Varmail`, `MDTest`, `Exim`, and `RocksDB` on six filesystems. Figure 15 illustrates the results. In `Varmail` (80 threads) workload, DJFS outperforms JBD(mem) by 4.3×, FastCommit by 4.5×, Z-Journal by 4.3×, iJournaling by 5.7×, and CJFS by 3.1×. We observe similar performance trends in `MDTest` and `Exim`. The performance differences are mainly due to the difference in the length of transaction lock-up interval and in the frequency of transaction conflicts.

Transaction Lock-Up Overhead. We measure the length of the transaction lock-up interval in each workload. Figure 16 illustrates the results. In all four workloads, DJFS and iJournaling yield the shortest transaction lock-up interval while FastCommit renders the longest one. In varmail workload, the average transaction lock-up intervals of FastCommit, JBD(mem), CJFS, Z-Journal, iJournaling, and DJFS are 211us, 120us, 65us, 39us, 0.2us, and 26us, respectively. When the filesystem defines multiple running transactions (DJFS, iJournaling, and Z-Journal), the transaction lock-up interval becomes shorter compared to when the filesystem defines only one running transaction (CJFS, FastCommit, and JBD(mem)). The length of the transaction lock-up interval is inversely proportional to the number of filesystem operations in a journal transaction. Among three journaling filesystems with multiple running transactions, iJournaling and DJFS yield shorter transaction lock-up interval than Z-Journal.

Transaction Conflict Overhead. We count the number of times where a transaction is blocked due to the transaction conflict. We call it a conflict count. When the transaction holds the shadow copy of the updated metadata, the subsequent transaction is not blocked even though it updates the same metadata. JBD(mem), FastCommit, and CJFS are free from the conflicts since they use the shadow copy of the updated data in journaling. Figure 17 illustrates the average conflict

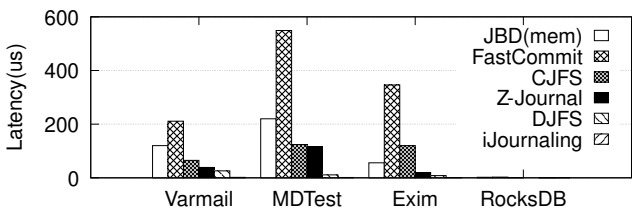


Figure 16: Transaction Lock-Up Time, 80 Threads

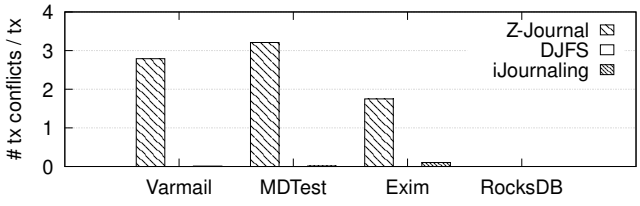
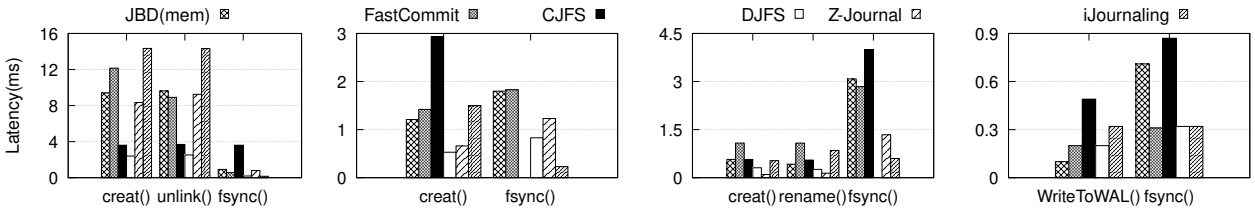


Figure 17: Number of conflicts per transaction



(a) `Varmail` (b) `MDTest` (c) `Exim` (d) `RocksDB-fillsync`

Figure 18: File Operation Latency : JBD, CJFS, Z-journal, FastCommit, and DJFS

counts per transaction in Z-Journal, iJournaling, and DJFS, respectively. In `Varmail` (80 threads), the average conflict count per transaction corresponds to 2.8 in Z-journal. DJFS and iJouranling are virtually free from any conflicts. In DJFS, a transaction is subject to conflict when an inode is referenced by more than one directory and when a block reuse occurs. This phenomenon is observed few times in workloads used in this study (§3.1).

Comparison with iJournaling. Per-directory transaction (DJFS) successfully carries more filesystem operations than per-file transaction (iJournaling) does. A transaction of DJFS carries 2.4× more file operations than a transaction of iJournaling on the average (Table 3), while they render a similar transaction commit latency. At `Varmail` and `RocksDB-fillsync`, the coalescing degree of iJournaling is less than 1. The iJournaling creates two transactions to create a file; one for a file and the other for the parent directory.

Log Record Creation Overhead. DJFS creates the shadow copy of the updated metadata when the file operation updates it as XFS [56] does. On the other hand, FastCommit creates the shadow copy of the updated metadata when the transaction commit starts. Thus, if the same metadata is updated multiple times till the journal commit, DJFS can create the shadow copy of the metadata multiple times, whereas FastCommit creates the shadow copy only once. While DJFS's approach can reduce the transaction lock-up time, it can require redundant memory copy. In `RocksDB`, DJFS performs 5% worse than FastCommit. We find that this is due to the overhead of creating the shadow copy in DJFS.

In-Memory File Operation. We examine the effect of the transaction lock-up and the transaction conflict on the filesystem operation. Figure 18 illustrates the results. The latency of the `creat()` of DJFS, JBD(mem), FastCommit, CJFS, iJournaling, and Z-Journal are 2.4ms, 9.4ms, 12.2ms, 3.6ms, 14.3ms, and 8.3ms, respectively (`Varmail`, 80 threads).

Workload	`Varmail`	`MDTest`	`Exim`	`fillsync`
DJFS	4.2	2.0	1.9	1.0
iJournaling	0.9	1.0	1.3	0.6

Table 3: Coalescing Degree (# of file op/tx)

We observe that the `creat()` latency is proportional to the transaction lock-up time (Figure 16) and the per-transaction conflict frequency (Figure 17). In iJournaling, we find that the transaction conflict caused by JBD, negatively impacts the latency even though its per-file transaction is free from the transaction lock-up and transaction conflict.

`fsync()` Latency. We examine the the effect of fine-grained journaling. As shown in Figure 18, the `fsync()` latencies of DJFS, iJournaling, JBD(mem), FastCommit, CJFS, and Z-Journal are 0.2ms, 0.16ms, 0.9ms, 0.56ms, 3.6, and 8.3ms, respectively (`Varmail`, 80 threads). The `fsync()` latency is proportional to the transaction size. The transaction sizes of DJFS, iJournaling, JBD(mem), FastCommit, CJFS, and Z-Journal are 1.6 KB, 5.2 KB, 18.3 KB, 1.8 KB, 324.0 KB, and 32.6 KB, respectively. We observe that FastCommit shows 2.8× longer latency than DJFS even if their transaction sizes are almost the same. The latency difference between DJFS and FastCommit is due to difference in the transaction lock-up time; FastCommit's transaction lock-up time is x8 longer than DJFS (Figure 16).

7.3 Memory Consumption of DJFS

Memory Consumption of Transaction Structures. One of the most important issues in using the per-directory transaction is its memory pressure. We measure the memory consumption of all transaction structures in the system (Table 4). Transaction structure is a data structure that represents a transaction excluding the log records. The size of the transaction structure is 240 Byte in DJFS. The amount of memory for transaction structures in DJFS ranges from 13.6 MB to 22.1 MB. It is two orders of magnitude larger than those of JBD and CJFS. Given the size of main memory in the server, we carefully believe that the memory pressure of the DJFS is within the tolerable range.

The transaction structures of the DJFS consumes larger memory than the others due to the checkpointing transaction. DJFS has a larger number of checkpointing transactions than the other filesystems do. In the other filesystems tested in our experiment, the checkpointing transaction is deallocated when all of its page cache entries are removed. In DJFS, a page cache entry is not removed from the checkpointing

transaction and therefore the checkpointing transaction structure is not deallocated till it is checkpointed. Among the four benchmarks we used, EXIM uses the largest number of dedicated directories, 352, whose memory pressure for transaction structures may be insignificant.

Memory Consumption of Log Records. We examine the total amount of memory for the log records (shadow copy of the updated metadata). The amount of memory for the log records is less than 250 KB, which we carefully believe is negligible. That is because the log records are deallocated when they are made durable at the journal region.

7.4 Crash Recovery

We verify the correctness of DJFS journaling using CrashMonkey [42]. We evaluate three specific scenarios—35, 106, and 321—from the `xfstests` suite. These tests focus on (i) `rename()`, which updates multiple directories, (ii) `link()` across directories, and (iii) repetitive executions of file system operations such as `creat()`, `unlink()`, `append()`, `mkdir()`, and `rmdir()`. Additionally, we include tests for renaming the root directory to a subdirectory (`rename_root_to_sub`) and for creating and deleting files (`create_delete`). DJFS passed all 1,000 test cases generated for each selected scenario. For recovery time, DJFS takes 2.5 seconds when a force shutdown is triggered at `Varmail`, 80 threads.

8 Discussion

`fsync()` semantics. The `fsync()` behavior of DJFS strictly follows the POSIX standard [59]; `fsync()` of DJFS ensures that all data associated with a given file descriptor is flushed to the storage device. This behavior is consistent with the `fsync()` of the widely used filesystems such as XFS, F2FS, and BTRFS. On the other hand, `fsync()` of EXT4 flushes not only the updated metadata of a given file but also all updated metadata in the system.

Performance under Cache Miss When all accesses to the CMM-H device renders the cache miss and are serviced from the flash, DJFS achieves 1.24× higher throughput than block granularity JBD in `Varmail`, 80 threads. When DJFS works as a plain block granularity journaling without cache-line granularity access, DJFS achieves 1.96× higher throughput than block granularity JBD under the same workload. When the cache miss occurs, the current CMM-H device loads the requested data from the flash to the cache and services the request from the cache. Due to cache management overhead, DJFS with cache-line granularity journaling exhibits worse performance than DJFS with plain block granularity journaling in the worst case. In worst case, DJFS still works better than JBD.

Workload	DJFS	Z-Journal	JBD	CJFS
`Varmail`	13.6 MB	17.3 MB	130.9 KB	188.1 KB
`MDTest`	10.9 MB	694.7 MB	1924.6 KB	972.8 KB
`Exim`	10.4 MB	152.2 MB	512.7 KB	354.5 KB
`RocksDB`	22.1 MB	Panic	1.5 KB	2.6 KB

Table 4: Memory Consumption of transaction structures

9 Related Work

Scalable Journal. IceFS [39] and SpanFS [30] divide the file system into multiple partitions and allocate the transaction in per-partition basis. Z-Journal [32] and MQFS [38] improve scalability by allocating the transaction in per-core basis. XFS [56] and CJFS [47] improve the scalability by eliminating file operation blocking due to conflicts.

Reduce Journal Commit Latency. iJournaling [48] and FastCommit [51] minimize transaction sizes by committing only file metadata. BarrierFS [62] and HORAE [37] reduce commit latency by leveraging order-preserving I/O stacks.

Filesystem for NVM. BPFS [24] introduces short-circuit shadow paging to update the filesystem at a fine granularity. NOVA [65] introduces per-inode logs. Aerie [61] proposes a user-space filesystem to exploit the low latency characteristics of NVM. SplitFS [29] and MadFS [68] handle the data update in the user space and the metadata update in the kernel space.

Heterogeneous Filesystems. Ziggurat [67], SPFS [63], and Strata [34] are heterogeneous filesystems with both NVM and SSD. Strata writes the data to the NVM first and migrates it to the SSD in the background. Ziggurat, Orthus, and SPFS choose the storage medium based on I/O characteristics.

10 Conclusion

We propose DJFS, a *Directory-granularity Journaling File System* for new memory semantic SSD, CMM-H. We identify three key challenges in realizing the per-directory transaction: (i) updating a file with multiple links, (ii) updating multiple directories in a transactional fashion, and (iii) ensuring the commit order among the transactions. We address these challenges by path-based transaction selection, transaction coalescing, and transaction conflict resolution. DJFS presents a way to exploit the dual mode access characteristics of the memory semantic SSD.

Acknowledgements We are deeply indebted to our shepherd, Huaicheng Li, for helping shape the final version of this paper. We are also grateful to the anonymous reviewers for their comments. We thank SMRC in Samsung Electronics for allowing the use of their data center facility for the testing. This work was funded by Samsung Electronics, IITP, Korea (RS-2024-00459026), and NRF, Korea (NRF-2020R1A2C3008525).

References

[1] Exim github. https://github.com/Exim/exim.git.

[2] Git. https://git-scm.com/.

[3] Marvell Introduces Breakthrough Structera CXL Product Line to Address Server Memory Bandwidth and Capacity Challenges in Cloud Data Centers. https://www.marvell.com/company/newsroom/marvell-introduces-breakthrough-structera-cxl-product-line-to-address-server-memory-bandwidth-and-capacity-challenges-in-cloud-data-centers.html.

[4] MDTest, lustre wiki. https://wiki.lustre.org/MDTest.

[5] Mercurial. https://www.mercurial-scm.org/.

[6] Micron Memory Expansion Module using CXL. https://www.micron.com/products/memory/cxl-memory.

[7] MySQL. https://www.mysql.com/.

[8] MySQL github. https://github.com/mysql/mysql-server/.

[9] RocksDB. https://rocksdb.org/.

[10] RocksDB github. https://github.com/facebook/rocksdb/.

[11] Samsung CXL Memory Module DRAM. https://semiconductor.samsung.com/next-gen-memory/cmm-d.

[12] SKHynix CXL Memory Module. https://product.skhynix.com/products/cxl/cxl.go.

[13] SQLite. https://www.sqlite.org/index.html.

[14] World's first CXL 2.0 and PCIe Gen5 Switch IC . https://www.xconn-tech.com/product.

[15] Ahmed Abulila, Vikram Sharma Mailthody, Zaid Qureshi, Jian Huang, Nam Sung Kim, Jinjun Xiong, and Wen-mei Hwu. Flatflash: Exploiting the byte-accessibility of ssds within a unified memory-storage hierarchy. In *Proc. of 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ACM ASPLOS'19)*.

[16] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, Minsik Cho, Carlo C Del Mundo, Mohammad Rastegari, and Mehrdad Farajtabar. LLM in a Flash: Efficient large language model inference with limited memory. *arXiv preprint arXiv:2312.11514*, 2023.

[17] Ravi Bhargava and Kai Troester. AMD Next Generation" Zen 4" Core and 4 th Gen AMD EPYC™ Server CPUs. *IEEE Micro*, 2024.

[18] Srivatsa S Bhat, Rasha Eqbal, Austin T Clements, M Frans Kaashoek, and Nickolai Zeldovich. Scaling a File System to Many Cores Using an Operation Log. In *Proc. of 26th ACM Symposium on Operating Systems Principles (ACM SOSP'17)*.

[19] Dhruba Borthakur et al. HDFS architecture guide. *Hadoop apache project*, 53(1-13):2, 2008.

[20] Silas Boyd-Wickizer, Austin T Clements, Yandong Mao, Aleksey Pesterev, M Frans Kaashoek, Robert Morris, and Nickolai Zeldovich. An analysis of linux scalability to many cores. In *Proc. of 9th USENIX Symposium on Operating Systems Design and Implementation (USENIX OSDI'10)*, 2010.

[21] Cheng Chen, Jun Yang, Qingsong Wei, Chundong Wang, and Mingdi Xue. Fine-grained metadata journaling on NVM. In *Proc. of 32nd Symposium on Mass Storage Systems and Technologies (IEEE MSST' 16)*.

[22] Vijay Chidambaram, Thanumalayan Sankaranarayana Pillai, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. Optimistic crash consistency. In *Proc. of 24th ACM Symposium on Operating Systems Principles (ACM SOSP'13)*.

[23] Vijay Chidambaram, Tushar Sharma, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. Consistency without ordering. In *Proc. of 10th USENIX Conference on File and Storage Technologies (USENIX FAST' 12)*.

[24] Jeremy Condit, Edmund B Nightingale, Christopher Frost, Engin Ipek, Benjamin Lee, Doug Burger, and Derrick Coetzee. Better I/O through byte-addressable, persistent memory. In *Proc. of 22nd ACM Symposium on Operating Systems Principles (ACM SOSP' 09)*.

[25] Mingkai Dong and Haibo Chen. Soft updates made simple and fast on non-volatile memory. In *Proc. of 2017 USENIX Annual Technical Conference (USENIX ATC' 17)*.

[26] Robert Hagmann. Reimplementing the Cedar file system using logging and group commit. In *Proc. of 11th ACM Symposium on Operating Systems Principles (ACM SOSP' 87)*.

[27] Philip Hazel. *Exim: The Mail Transfer Agent*. " O'Reilly Media, Inc.", 2001.

[28] Yige Hu, Zhiting Zhu, Ian Neal, Youngjin Kwon, Tianyu Cheng, Vijay Chidambaram, and Emmett Witchel. TxFS: Leveraging File-System Crash Consistency to

Provide ACID Transactions. In *Proc. of 2018 USENIX Annual Technical Conference (USENIX ATC'18)*.

[29] Rohan Kadekodi, Se Kwon Lee, Sanidhya Kashyap, Taesoo Kim, Aasheesh Kolli, and Vijay Chidambaram. SplitFS: Reducing software overhead in file systems for persistent memory. In *Proc. of 27th ACM Symposium on Operating Systems Principles (ACM SOSP' 19)*.

[30] Junbin Kang, Benlong Zhang, Tianyu Wo, Weiren Yu, Lian Du, Shuai Ma, and Jinpeng Huai. SpanFS: A Scalable File System on Fast Storage Devices. In *Proc. of 2015 USENIX Annual Technical Conference (USENIX ATC'15)*.

[31] Dohyun Kim, Kwangwon Min, Joontaek Oh, and Youjip Won. ScaleXFS: Getting scalability of XFS back on the ring. In *Proc. of 20th USENIX Conference on File and Storage Technologies (USENIX FAST'22)*.

[32] Jongseok Kim, Cassiano Campes, Joo-Young Hwang, Jinkyu Jeong, and Euiseong Seo. Z-Journal: Scalable Per-Core Journaling. In *Proc. of 2021 USENIX Annual Technical Conference (USENIX ATC'21)*.

[33] Miryeong Kwon, Sangwon Lee, and Myoungsoo Jung. Cache in Hand: Expander-Driven CXL Prefetcher for Next Generation CXL-SSD. In *Proc. of 15th ACM Workshop on Hot Topics in Storage and File Systems (ACM HotStorage' 23)*.

[34] Youngjin Kwon, Henrique Fingler, Tyler Hunt, Simon Peter, Emmett Witchel, and Thomas Anderson. Strata: A cross media file system. In *Proc. of 26th ACM Symposium on Operating Systems Principles (ACM SOSP'17)*.

[35] Changman Lee, Dongho Sim, Jooyoung Hwang, and Sangyeun Cho. F2fs: A new file system for flash storage. In *Proc. of 13th USENIX Conference on File and Storage Technologies (USENIX FAST' 15)*.

[36] Eunji Lee, Hyokyung Bahn, and Sam H Noh. Unioning of the buffer cache and journaling layers with non-volatile memory. In *Proc. of 11th USENIX Conference on File and Storage Technologies (USENIX FAST' 13)*.

[37] Xiaojian Liao, Youyou Lu, Erci Xu, and Jiwu Shu. Write dependency disentanglement with HORAE. In *Proc. of 14th USENIX Symposium on Operating Systems Design and Implementation (USENIX OSDI' 20)*.

[38] Xiaojian Liao, Youyou Lu, Zhe Yang, and Jiwu Shu. Crash Consistent Non-Volatile Memory Express. In *Proc. of 28th ACM Symposium on Operating Systems Principles (ACM SOSP'21)*.

[39] Lanyue Lu, Yupu Zhang, Thanh Do, Samer Al-Kiswany, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. Physical Disentanglement in a Container-Based File System. In *Proc. of 11th USENIX Symposium on Operating Systems Design and Implementation (USENIX OSDI'14)*.

[40] Richard McDougall and Jim Mauro. Filebench, 2005.

[41] Chandrasekaran Mohan, Don Haderle, Bruce Lindsay, Hamid Pirahesh, and Peter Schwarz. ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging. *ACM Transactions on Database Systems (TODS)*, 17(1):94–162, 1992.

[42] Jayashree Mohan, Ashlie Martinez, Soujanya Ponnapalli, Pandian Raju, and Vijay Chidambaram. Finding Crash-Consistency Bugs with Bounded Black-Box Crash Testing. In *Proc. of 13th USENIX Symposium on Operating Systems Design and Implementation (USENIX OSDI' 18)*.

[43] Ashley O Munch, Nevine Nassif, Carleton L Molnar, Jason Crop, Rich Gammack, Chinmay P Joshi, Goran Zelic, Kambiz Munshi, Min Huang, Charles R Morganti, et al. 2.3 Emerald Rapids: 5th-Generation Intel Xeon Scalable Processors. In *2024 IEEE International Solid-State Circuits Conference (IEEE ISSCC'24)*, volume 67, pages 40–42. IEEE, 2024.

[44] Nevine Nassif, Ashley O Munch, Carleton L Molnar, Gerald Pasdast, Sitaraman V Lyer, Zibing Yang, Oscar Mendoza, Mark Huddart, Srikrishnan Venkataraman, Sireesha Kandula, et al. Sapphire rapids: The next-generation intel xeon scalable processor. In *2022 IEEE International Solid-State Circuits Conference (IEEE ISSCC'22)*, volume 65, pages 44–46. IEEE, 2022.

[45] NVM Express. What is Persistent Memory Region? https://nvmexpress.org/faq-items/what-is-persistent-memory-region/.

[46] Joontaek Oh, Sion Ji, Yongjin Kim, and Youjip Won. exF2FS: Transaction Support in Log-Structured Filesystem. In *Proc. of 20th USENIX Conference on File and Storage Technologies (USENIX FAST'22)*.

[47] Joontaek Oh, Seung Won Yoo, Hojin Nam, Changwoo Min, and Youjip Won. CJFS: Concurrent Journaling for Better Scalability. In *Proc. of 21st USENIX Conference on File and Storage Technologies (USENIX FAST'23)*.

[48] Daejun Park and Dongkun Shin. iJournaling:Fine-Grained Journaling for Improving the Latency of Fsync System Call. In *Proc. of 2017 USENIX Annual Technical Conference (USENIX ATC'17)*.

[49] Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C Arpaci-Dusseau, and Remzi H Arpaci-Dusseau. All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications. In *Proc. of 11th USENIX Symposium on Operating Systems Design and Implementation (USENIX OSDI'14)*.

[50] Samsung MSL. CXL Memory Module - Hybrid. https://semiconductor.samsung.com/news-events/tech-blog/samsung-cxl-solutions-cmm-h/.

[51] Harshad Shirwadkar, Saurabh Kadekodi, and Theodore Tso. FastCommit: resource-efficient, performant and cost-effective file system journaling. In *Proc. of 2024 USENIX Annual Technical Conference (USENIX ATC' 24)*.

[52] Dimitrios Skarlatos, Umur Darbaz, Bhargava Gopireddy, Nam Sung Kim, and Josep Torrellas. Babelfish: Fusing address translations for containers. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ACM/IEEE ISCA'20)*, pages 501–514. IEEE, 2020.

[53] Yongseok Son. Design and Implementation of a Log-structured Buffer Based on Non-volatile Memory. *Journal of KIISE*, 45(11):1117–1123, 2018.

[54] Yongseok Son, Sunggon Kim, Heon Y Yeom, and Hyuck Han. High-Performance Transaction Processing in Journaling File Systems. In *Proc. of 16th USENIX Conference on File and Storage Technologies (USENIX FAST' 18)*.

[55] Hyunsub Song, Shean Kim, J Hyun Kim, Ethan JH Park, and Sam H Noh. First responder: Persistent memory simultaneously as high performance buffer cache and storage. In *Proc. of 2021 USENIX Annual Technical Conference (USENIX ATC' 21)*.

[56] Adam Sweeney, Doug Doucette, Wei Hu, Curtis Anderson, Mike Nishimoto, and Geoff Peck. Scalability in the XFS File System. In *Proc. of 1996 USENIX Annual Technical Conference (USENIX ATC' 96)*.

[57] Thanumalayan Sankaranarayana Pillai and Ramnatthan Alagappan and Lanyue Lu and Vijay Chidambaram and Andrea C. Arpaci-Dusseau and Remzi H. Arpaci-Dusseau. Application Crash Consistency and Performance with CCFS. In *Proc. of 15th USENIX Conference on File and Storage Technologies (USENIX FAST'17)*.

[58] The Open Group. IEEE Std 1003.1™-2017. https://pubs.opengroup.org/onlinepubs/9699919799/functions/rename.html.

[59] The Open Group. IEEE Std 1003.1™-2017. https://pubs.opengroup.org/onlinepubs/9699919799/.

[60] Stephen C Tweedie et al. Journaling the Linux ext2fs Filesystem. In *The Fourth Annual Linux Expo*.

[61] Haris Volos, Sanketh Nalli, Sankarlingam Panneerselvam, Venkatanathan Varadarajan, Prashant Saxena, and Michael M Swift. Aerie: Flexible file-system interfaces to storage-class memory. In *Proc. of 9th European Conference on Computer Systems (ACM EuroSys' 14)*.

[62] Youjip Won, Jaemin Jung, Gyeongyeol Choi, Joontaek Oh, Seongbae Son, Jooyoung Hwang, and Sangyeun Cho. Barrier-Enabled IO Stack for Flash Storage. In *Proc. of 16th USENIX Conference on File and Storage Technologies (USENIX FAST'18)*.

[63] Hobin Woo, Daegyu Han, Seungjoon Ha, Sam H Noh, and Beomseok Nam. On Stacking a Persistent Memory File System on Legacy File Systems. In *Proc. of 21st USENIX Conference on File and Storage Technologies (USENIX FAST'23)*.

[64] Haitao Wu, Guohan Lu, Dan Li, Chuanxiong Guo, and Yongguang Zhang. MDCube: a high performance network structure for modular data center interconnection. In *Proc. of 5th international conference on Emerging networking experiments and technologies (ACM CoNEXT'09)*, pages 25–36, 2009.

[65] Jian Xu and Steven Swanson. NOVA: A Log-Structured File System for Hybrid Volatile/Non-volatile Main Memories. In *Proc. of 14th USENIX Conference on File and Storage Technologies (USENIX FAST'16)*.

[66] Shao-Peng Yang, Minjae Kim, Sanghyun Nam, Juhyung Park, Jin-yong Choi, Eyee Hyun Nam, Eunji Lee, Sungjin Lee, and Bryan S Kim. Overcoming the Memory Wall with CXL-Enabled SSDs. In *Proc. of 2023 USENIX Annual Technical Conference (USENIX ATC' 23)*.

[67] Shengan Zheng, Morteza Hoseinzadeh, and Steven Swanson. Ziggurat: A Tiered File System for Non-Volatile Main Memories and Disks. In *Proc. of 17th USENIX Conference on File and Storage Technologies (USENIX FAST'19)*.

[68] Shawn Zhong, Chenhao Ye, Guanzhou Hu, Suyan Qu, Andrea Arpaci-Dusseau, Remzi Arpaci-Dusseau, and Michael Swift. MadFS:Per-File Virtualization for Userspace Persistent Memory Filesystems. In *Proc. of 21st USENIX Conference on File and Storage Technologies (USENIX FAST' 23)*.

[69] Diyu Zhou, Vojtech Aschenbrenner, Tao Lyu, Jian Zhang, Sudarsun Kannan, and Sanidhya Kashyap. Enabling High-Performance and Secure Userspace NVM File Systems with the Trio Architecture. In *Proc. of 29th ACM Symposium on Operating Systems Principles (ACM SOSP' 23)*.